

1. 시스템 개요

1.1. 통신 미들웨어 및 하드웨어 구성

- 통신 미들웨어: MQTT
- 하드웨어 구성
 - 파이 1: 외부(센서)
 - PIR
 - 조도 센서(+ MCP)
 - 파이 2: 내부(액추에이터)
 - LED
 - 초음파
 - 서보 모터
 - 스위치

1.2 목표

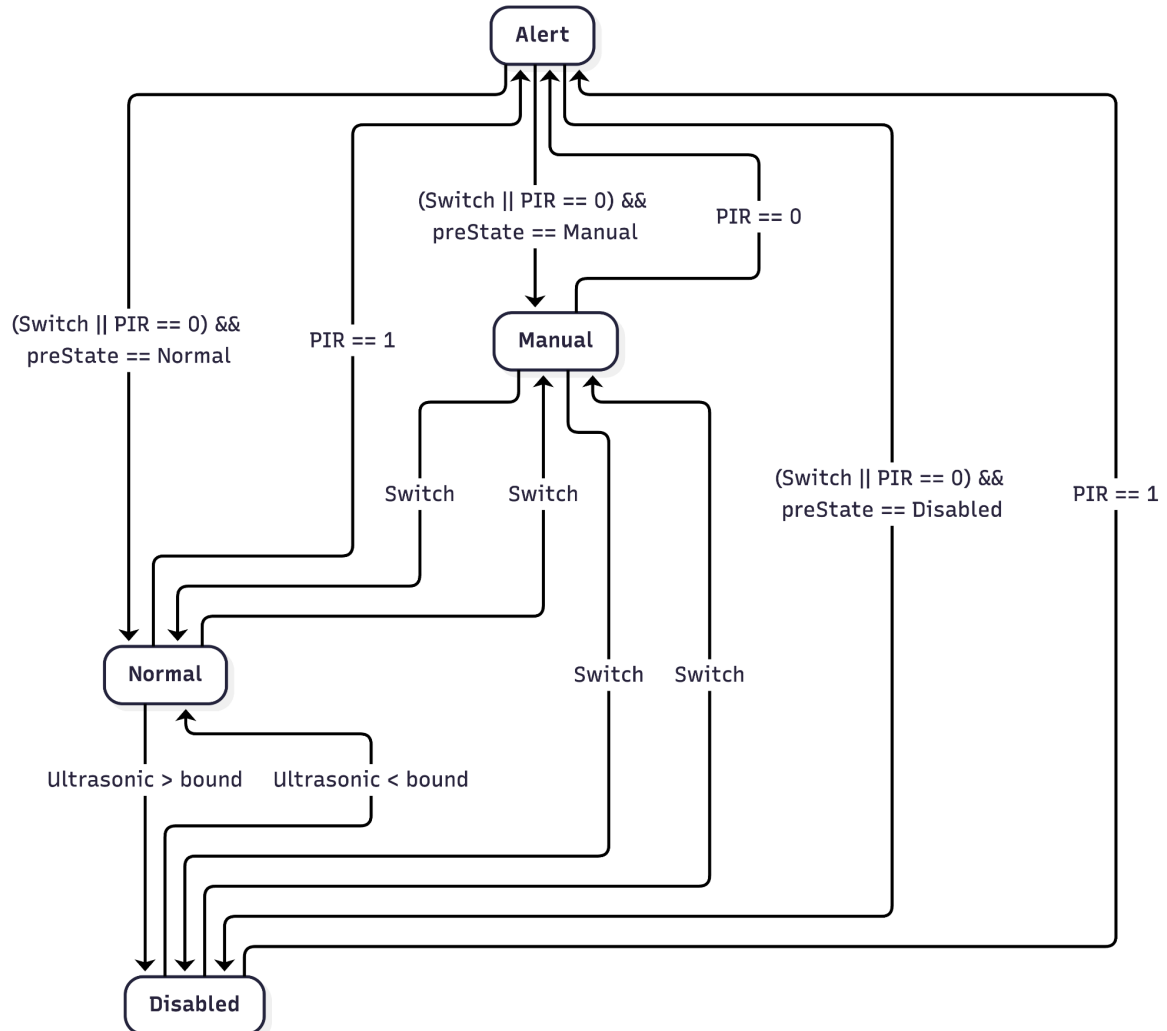
- 서보모터가 건물의 외벽 루버나 블라인드의 각도를 미세하게 조절하여, 에너지 효율을 극대화하고 실내환경을 쾌적하게 유지하는 것을 목적으로 한다.

2. 주요 시나리오

- 빛이 들어오는 각도가 바뀔 때, 블라인드 각도 조절
 - 조도 센서와 A/D 컨버터(MCP3208)를 활용하여 광량 감지
 - 빛이 들어오는 각도가 바뀌었을 때(광량이 일정 수준을 초과하였을 때) 서보 모터를 통해 블라인드 각도 조절
- 수동 제어(스위치 버튼을 통한 제어)
 - 스위치 버튼을 활용해 클릭 감지
 - 현재 침입 감지 상태인지 / 블라인드가 열려있는지, 닫혀있는지 판단
 - 침입 감지 상태라면: 이전 상태로 전환
 - 침입 감지 상태가 아니라면: 서보모터 동작으로 닫도록 제어, 수동 제어 상황으로 전환
 - 상황 종료 전까지 LED 조명을 황색(수동 강제 제어 상황) 점멸
 - 상황 종료 전파:
 - 스위치 재클릭 시 상황 종료
- 초음파 센서에 걸릴 때, (물체로 동작이 불가능한 상황)
 - 초음파 센서에 의해 동작 범위내의 물체 감지.
 - 동작 불가 상태 전환: 모터의 제어를 중지
 - LED 점등 유지: 물체가 없어질 때까지
- 바깥쪽에서 접근이 있을때(침입 감지)
 - PIR 센서를 통해 접근 감지
 - 서보모터에 닫힘 상태로 전환하기 트리거
 - LED 조명을 점멸: 감지된 동안 지속
 - 상황 종료 전파: 수동 스위치를 눌러 개방.
 - 수동 스위치 누르면 평소 모드로 전환되면 다시 이전 상태로.

- PIR 센서에서 감지가 더이상 되지 않는 경우 또한 이전 상태로.

2.1. State 정의



- **Normal:** 빛의 세기에 따라 블라인드를 조절하며 일정 수준의 세기 이상으로 조절하는 모드
 - 빛의 세기가 임계점에 도달할 때까지 블라인드를 연다.
 - 예시: 밤의 경우, 수동 제어가 없다면 무조건 전체 개방될 가능성 높음.
- **Manual:** 블라인드를 모두 닫음.
 - 수동 스위치를 누름으로써 발생되고 수동 스위치를 눌러 종료할 수 있음.
- **Disabled:** 동작 범위 내의 물체가 감지되어 이용자의 개입이 요구되는 상황
- **Alert:** 외부의 접근 발생
-

2. 모듈 정의

2.1. 파이 1

- 조도 센서 디바이스 드라이버(kernel module)
 - Input: u8[3] (unsigned char)
(MCP3208으로부터 수신되는 SPI 3바이트 디지털 데이터 (물리 신호))
 - Output: 조도값 문자열
(유저 공간에서 read() 함수를 호출 시 반환. 범위: "0\n" ~ "4095\n")
(MCP3208한테서 정수로 받아오지만 디버깅 편의를 위해 문자열로 변환)
- PIR 센서 디바이스 드라이버
 - Input: int (gpio_get_value() 반환값)
(PIR 핀의 전압 변화(High/Low)(물리 신호), IRQ 발생 시 핸들러에서 읽어들임)
 - Output: 침입 상태값 문자열
(유저 공간에서 read() 호출 시 반환) (침입 감지 "1\n", 평시 "0\n")
(조도 드라이버와 동일한 이유로 문자열로 변환)
 - IRQ
 - IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING
: 전압 상승/하강 양쪽 엣지 감지
 - spin_lock_irqsave
: IRQ 핸들러와 read() 간 공유 자원 보호
- 데이터 취합 모듈(user application)
 - Input:
 - 조도값 문자열
(조도 드라이버에서 read() 해옴. 이후 atoi()로 정수 변환)
 - 침입 상태값 문자열
(PIR 드라이버에서 read() 해옴. 이후 atoi()로 정수 변환)
 - Output: json 문자열 (MQTT 브로커에 publish함)
 - 예: {"light_a": 2000, "light_b": 1000, "intruder": 1}

2.2. 파이 2

- 최종 서보모터 각도 제어 모듈(user application)
 - 각도 조절 15도 (추후 수정 예정)
 - MQTT로 데이터를 받아 동작 각도를 계산하고 서보 모터 디바이스 드라이버에 write 수행
 - Input: MQTT로 전달받은 목표 각도 값
 - Output: 서보모터를 지정된 각도로 변환하기 위한 하드웨어 PWM 펄스 폭
- 초음파 센서 디바이스 드라이버(kernel module)
 - Input: 반사되어 돌아오는 초음파
 - Output: 계산된 장애물 거리
 - IRQ(엣지 트리거 인터럽트)
- 서보 모터 디바이스 드라이버(kernel module)

- LED 제어(kernel module)
 - Input: 현재 상태를 정의한 신호(평시 상태 0, 수동 제어 상태1, 동작 불가 상태2, 침입 감지 상태3 으로 정의?)
 - Output: 상태에 맞는 색 LED(GPIO) High/Low 전환
- 스위치 디바이스 드라이버(kernel module)
 - Input: 물리 버튼이 눌렸을 때 발생하는 GPIO 하강/상승 에지 인터럽트?
 - Output: 유저 앱이 읽어갈 수 있는 스위치 토글 상태
 - IRQ

3. 기술적 도전사항

- Race Condition 방지를 위한 커널-유저 공간 및 모듈 동기화
 - 과제: 스위치, PIR, 초음파 센서 감지가 동시다발적으로 발생할 때 메인 로직 보장
 - 예: 스위치를 누를 때와 외부 조도 데이터 수신이 겹칠 때, 공유 자원 (시스템 상태)에 동시 접근하는 경쟁 상태 발생
- 조도 센서 값에 따른 서보 모터 미세 조정
 - 과제: MCP3208을 통해 들어온 미세한 조도 값을 바탕으로 서보 모터 각도 조절을 부드럽게 매핑하여 진행

4. 모듈 파트 분배

4.1. 노을영

- 최종 서보모터 각도 제어 모듈(user application) – 노을영
- 서보 모터 디바이스 드라이버(kernel module) – 노을영

4.2. 박민주

- 초음파 센서 디바이스 드라이버(kernel module) – 박민주
- LED 제어(kernel module) – 박민주
- 스위치 디바이스 드라이버(kernel module) – 박민주

4.3. 조상현

- 조도 센서 디바이스 드라이버(kernel module) – 조상현
- PIR 센서 디바이스 드라이버 – 조상현
- 데이터 취합 모듈(user application) – 조상현

5. 구현 계획

- 5/25~31: 개별 구현

- 6/1~6/2: 상호 리뷰 및 점검
- 6/5 시스템 통합 & 자체 시연
- 이후 추가 개선 및 완성

궁금증?

모터 기준 모드별 UseCase?

normal모드 시나리오

1. pi1에서 조도센서가 값을 읽어오다 임계값 넘으면 MQTT로 서보모터에 열것인지 닫을것인지 전파
2. MQTT로 서보모터에서 받으면 들어온 방향에 맞게 15도 회전 명령을 pi2 커널 드라이버로 내림.
3. 드라이버가 받으면 Extern_simbol(state)에 따라 회전을 시킬지 말지 결정
=> 이론상 서보모터가 돌기 전까지 계속 시그널 발생시키니까 MQTT 전파 이후 조도센서 탐지에 딜레이를 주던가, 서보모터가 돌고 난 후 MQTT로 동작 재개를 위해 재전파 시켜야함

alert 시나리오

1. pi1에서 PIR 센서가 감지되면 PI 1의 조도센서 동작을 멈추고, Pi 2에 MQTT로 서보모터 닫힘 명령을 내린다.
2. pi 2는 MQTT를 받아 드라이버로 이전 서보모터 각도를 저장한 뒤 서보모터 닫힘 명령을 수행시킨다. state를 alert로 전환한다.
3. pi1에서 PIR 센서가 LOW로 바뀌면 MQTT로 서보모터 복구 명령을 내린다. PI 1의 조도센서 동작을 재개한다.
4. pi2는 MQTT를 받아 드라이버로 서보모터 각도 복구 동작을 수행시킨다. state도 복구한다

manual 시나리오

1. pi2에 연결된 스위치에 버튼이 눌리면 드라이버로 서보모터 닫힘 명령을 내린다.
2. pi 2는드라이버로 이전 서보모터 각도를 저장한 뒤 서보모터 닫힘 명령을 수행시킨다. state를 manual로 전환한다.
3. 스위치가 다시 눌리면 서보모터 각도복구 동작을 수행한다. state도 복구한다.
=> Pi1은 state를 몰라서 계속 동작할 수 있음. 괜찮은 건가? MQTT로 state가 변경되었다고 전파해야하나? 스위치도 차라리 pi1으로?

Disabled 시나리오

1. pi2에 연결된 ultrasonic에 감지되면 Extern_simbol(state)를 Disabled로 변경한다.
=> manual 시나리오랑 같은 문제